Malware Analysis - Lab Notes

IMPORTANT WARNING

- These notes have been made starting from my notes taken during lessons and the power points published by the professor, then rearranged through the help of Claude to produce a narrative flow.
- All the content belongs to the Professor of the course.
- While I try to ensure everything is inside these notes, please don't use them as a "source of truth", and study from official sources as well.

x86 Assembly for Malware Analysis — Reading Compiled C

Reverse engineering a malicious binary almost always begins with the same problem: the source code is gone, and all that remains is machine code. Before any payload, persistence mechanism, or network protocol can be understood, an analyst must be able to look at a stream of x86 instructions and reconstruct the high-level logic the compiler started from. The skill is pattern recognition. Compilers are deterministic translators, so every `if`, every `while`, and every function call leaves a recognizable fingerprint in the assembly. Learning those fingerprints on clean, well-understood code is what makes it possible to read hostile code later, where there are no comments, no symbols, and often deliberate attempts to mislead.

This chapter builds that foundation using a small 32-bit Windows program compiled from a set of ordinary C control-flow functions. The binary targets the 80386 instruction set — the first 32-bit Intel CPU and still the baseline for a huge amount of real-world malware — and is examined statically in a disassembler.

1. The Analysis Environment

Malware analysis is performed inside a disposable virtual machine, restored from a clean snapshot before each session. The guiding principle is isolation: the machine should contain nothing of value and should have no casual path to the outside world. A sample that is being dissected must never be given the opportunity to reach the host, the local network, or the internet, because even a sample intended only for study can execute, spread, or exfiltrate if handled carelessly. Keeping the

VM behind a restrictive network configuration and treating every binary as live ordnance is the baseline discipline of the field.

The primary tool is a disassembler. A **disassembler** translates raw machine-code bytes back into human-readable assembly mnemonics; a full-featured one such as IDA goes further, performing automatic analysis to recover function boundaries, cross-references, and data types, and it also embeds a **debugger** so the same binary can later be executed one instruction at a time. Static disassembly (reading the code without running it) and dynamic debugging (watching it run) are the two complementary halves of analysis. A free edition of IDA is sufficient for everything in this chapter.

1.1 Loading a Portable Executable

When a Windows binary is opened, the disassembler first identifies the file format. A 32-bit Windows program is reported as a *Portable Executable for 80386 (PE)*. Because nothing in the raw code declares how functions receive their arguments, the tool must assume a default **calling convention**, and for this platform that default is `__cdecl` — the convention used by virtually all C code compiled for 32-bit Windows. This assumption drives how the tool interprets stack accesses as function arguments, so it matters from the very first instruction.

1.2 Symbols: a luxury that real malware denies

A compiled binary can carry **debug information** — a side table mapping machine addresses back to original function names, variable names, and line numbers. The GCC/MinGW toolchain emits this information in the **DWARF** format (the standard used by Linux compilers, and by GCC even when it targets Windows). When such information is present, the disassembler offers to load it, and the result is dramatic: functions appear with their original names (`fun_if` , `fun_while` , `main`) and global variables appear as `A` , `B` , `C` instead of bare addresses.

This convenience is almost never available in practice. An author of malicious code has every incentive to **strip** the binary, removing all symbols so that functions appear only as generic labels like `sub_401460` and variables only as hexadecimal addresses. The same program can be built in both a symbol-rich form and a stripped form; comparing them is the fastest way to appreciate how much interpretive work symbols do, and how much of that work falls back onto the analyst when they are absent. For this reason, the patterns taught below must

ultimately be recognizable *without* any names at all — the names are training wheels.

There is one recurring exception worth remembering: a careless author sometimes leaves behind the path to the program database (PDB) file that was generated at build time. That leftover string can reveal the directory structure, user name, and project layout of the machine on which the malware was compiled — a small intelligence gift from a sloppy adversary.

1.3 Annotations are not code

The disassembler decorates its output with inferred commentary: reconstructed function prototypes, argument names, stack-frame summaries, and structural hints such as `; __unwind {`. These lines are **analysis output, not instructions that exist in the binary**. One of the first discriminations an analyst must learn is separating what the CPU will actually execute from what the tool merely believes about it. A related discrimination is separating the handful of functions the author wrote from the large body of **compiler and runtime support code** that every C program is linked against (startup routines, C library internals, thread-local-storage callbacks). Time is wasted analyzing library code; identifying and ignoring it is a skill in itself.

Finally, the disassembler offers two views of the same code. The **text view** is a linear listing; the **graph view** arranges the code into **basic blocks** — straight-line runs of instructions with a single entry and a single exit — connected by arrows that represent possible control flow. A conditional branch produces two outgoing edges, conventionally drawn so that the *condition-true* and *condition-false* successors are visually distinct. The graph view is the natural way to see the shape of an `if` or a loop at a glance, and it is where the constructs below become obvious.

2. Anatomy of a Function

Everything that follows depends on understanding how a function uses the **stack**, so this section establishes the model once.

2.1 Global variables

The program declares three global integers, `A`, `B`, and `C`. Globals live at fixed addresses in a data section for the entire lifetime of the process, so every reference to them is an absolute memory access. In symbol-rich disassembly these appear as

`ds:A`, `ds:B`, `ds:C`; stripped, they would appear as `ds:dword_407074` and similar. The key recognition cue is the *absolute address* operand — a hallmark of a global, as opposed to the frame-relative operands used for locals.

2.2 The stack frame, prologue, and epilogue

Local variables and function arguments are not stored at fixed addresses; they live on the stack, in a region called the function's **stack frame**. To address that region reliably, a function anchors it with the **base pointer** register `ebp`. Nearly every function therefore begins with the same two-instruction **prologue**:

```
push    ebp             ; save the caller's base pointer
mov     ebp, esp        ; set our base pointer to the current
stack top
```

The `push` writes the caller's `ebp` to the stack (growing the stack downward by four bytes) so it can be restored later; the `mov` then freezes `ebp` at the current top of stack. From this instant, `ebp` is a stable reference point for the whole function, even as `esp` moves around. A function that needs space for locals then subtracts from `esp` (for example `sub esp, 0x10` reserves sixteen bytes).

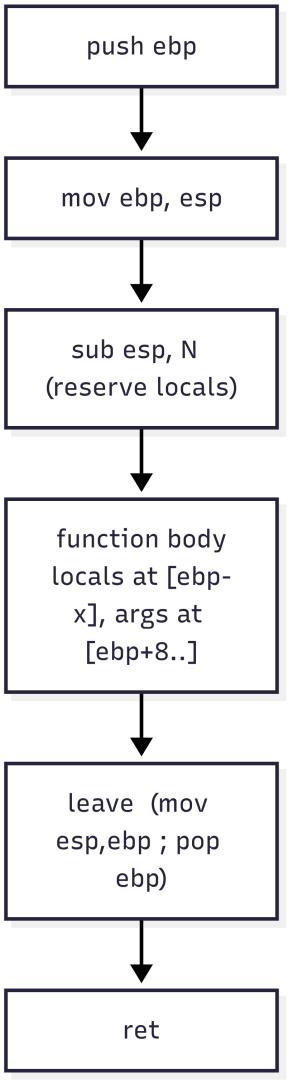
With `ebp` fixed, the two sides of the frame have opposite signs:

- **Local variables** sit *below* the base pointer, at **negative** offsets: `[ebp-4]`, `[ebp-8]`, `[ebp-0xC]`, and so on. A variable named by convention `var_4` is simply "the variable at offset 4 below `ebp`." Because these accesses are everywhere, the pattern `[ebp - something]` should immediately read as "local variable" even before its meaning is known. The disassembler lets the analyst assign a name and a size (a 32-bit integer is a `dword`) to each such slot and have every reference updated at once — a core manual-annotation step.
- **Arguments and the return address** sit *above* the base pointer, at **positive** offsets. Immediately after the prologue, `[ebp]` holds the saved old `ebp`, `[ebp+4]` holds the return address pushed by the `call` that entered the function, and `[ebp+8]`, `[ebp+0xC]`, `[ebp+0x10]` hold the first, second, and third arguments respectively.

The mirror image of the prologue is the **epilogue**, which tears the frame down and returns:

```
leave                   ; equivalent to: mov esp, ebp ; pop ebp
ret                     ; pop return address and jump to it
```

The single-byte `leave` instruction restores `esp` and `ebp` in one step; `ret` then transfers control back to the caller. A function that allocated no locals may instead end with a bare `pop ebp` / `ret`. Recognizing prologue and epilogue is what lets an analyst carve an unknown binary into functions in the first place.



2.3 The `__cdecl` calling convention

Under `__cdecl`, arguments are passed on the stack and pushed by the **caller** in right-to-left order, so that the leftmost argument ends up at the lowest address (`[ebp+8]`) inside the callee. Crucially, under `__cdecl` the **caller is responsible for cleaning the arguments off the stack** after the call returns. The function's integer result is returned in the `eax` register. These three facts — arguments on the stack, caller-side cleanup, result in `eax` — are the signature of `__cdecl`, and

because it is the default for this platform, they describe almost every hand-written function here.

3. The Driver: `main`

The program's `main` is a thin driver that sets the globals and calls each demonstration function in turn. Its disassembly establishes several patterns at once:

```
push    ebp
mov     ebp, esp
and     esp, 0FFFFFFF0h     ; align stack to a 16-byte boundary
call    ___main             ; MinGW C-runtime initialization
mov     ds:A, 1Eh           ; A = 30
mov     ds:B, 15h           ; B = 21
mov     ds:C, 3             ; C = 3
call    _fun_if
call    _fun_if_else
call    _fun_while
call    _fun_do_while
call    _fun_for
mov     eax, 0              ; return 0
leave
retn
```

Two details deserve comment. The instruction `and esp, 0FFFFFFF0h` clears the low four bits of the stack pointer, forcing it to a multiple of sixteen; this is **stack alignment**, required by the platform's ABI for performance and for certain instructions, and it is a normal part of a `main` prologue rather than anything suspicious. The call to `___main` is **MinGW runtime initialization** — compiler support code, not program logic — and is exactly the kind of boilerplate the analyst learns to recognize and step past. The constants `1Eh`, `15h`, and `3` are the decimal values 30, 21, and 3 written in hexadecimal, the native radix of disassembly; fluent hex reading is non-negotiable.

4. Recognizing Control-Flow Constructs

The heart of the chapter is translating conditional and iterative C back out of assembly. Assembly has no `if` or `while`; it has only **comparisons** that set the

`EFLAGS` register and **conditional jumps** that act on those flags. The compiler builds every high-level construct from this pair, and it does so following consistent templates.

The single most important idea is **condition inversion**. High-level code reads *"if the condition holds, execute the body."* The compiler finds it more convenient to generate *"if the condition does **not** hold, jump **past** the body."* As a result, the comparison in the assembly almost always tests the **logical opposite** of the condition written in C: a source-level `if (A < x)` becomes an assembly `jge` (jump if greater-or-equal) that *skips* the body. Reading the jump and mentally negating it to recover the original condition is a reflex the analyst must build.

4.1 The `if` statement — `fun_if`

The source computes a simple guarded subtraction:

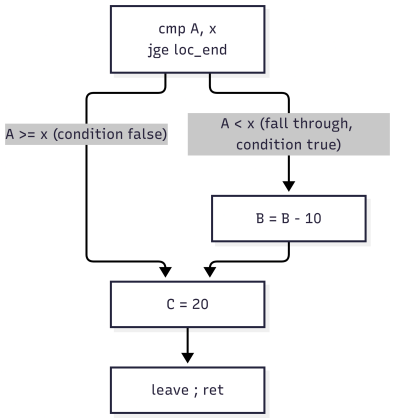
```
void fun_if() {
    int x = 30;
    if (A < x) {
        B = B - 10;
    }
    C = 20;
}
```

Its disassembly:

```
push    ebp
mov     ebp, esp
sub     esp, 10h
mov     [ebp-4], 1Eh        ; x = 30            (local var_4)
mov     eax, ds:A
cmp     eax, [ebp-4]        ; compare A with x
jge     short loc_end       ; if A >= x, skip the body   <--
inverted condition
mov     eax, ds:B           ; --- body: B = B - 10 ---
sub     eax, 0Ah
mov     ds:B, eax
loc_end:
mov     ds:C, 14h           ; C = 20
```

```
nop
leave
ret
```

The local `x` is materialized at `[ebp-4]` and set to `30`. The comparison `cmp eax, [ebp-4]` evaluates `A - x` only to set flags. The source condition is `A < x`; the compiler emits its negation, `jge` (jump if `A >= x`), to **jump over** the body when the body should *not* run. If the jump is not taken, control falls through into the body, which loads `B`, subtracts ten (`0Ah`), and stores it back. Either way, execution converges on the final `C = 20` (`14h`) and the epilogue. This convergence — two paths meeting at a common successor — is the structural signature of an `if`.



4.2 The `if / else` statement — `fun_if_else`

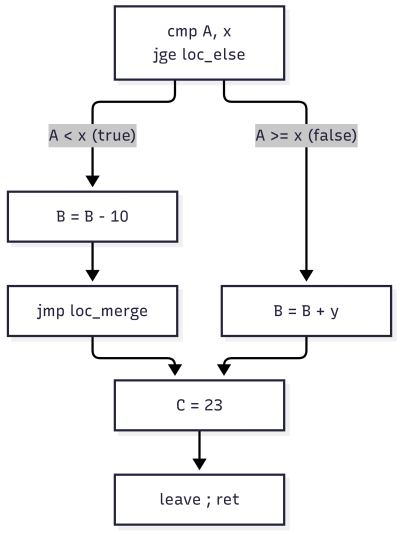
Adding an `else` introduces a second body and a mandatory unconditional jump:

```
void fun_if_else() {
    int x = 30, y = 5;
    if (A < x) {
        B = B - 10;
    } else {
        B = B + y;
    }
    C = 23;
}
```

```
sub     esp, 10h
mov     [ebp-4], 1Eh        ; x = 30
mov     [ebp-8], 5          ; y = 5
```

```
mov     eax, ds:A
cmp     eax, [ebp-4]
jge     short loc_else      ; if A >= x, go to the else branch
mov     eax, ds:B           ; --- then: B = B - 10 ---
sub     eax, 0Ah
mov     ds:B, eax
jmp     short loc_merge     ; skip over the else branch
loc_else:
mov     edx, ds:B           ; --- else: B = B + y ---
mov     eax, [ebp-8]
add     eax, edx
mov     ds:B, eax
loc_merge:
mov     ds:C, 17h           ; C = 23
```

The inverted `jge` now selects between two real alternatives rather than skipping one. The decisive new element is the `jmp short loc_merge` at the end of the *then* branch: once the true path finishes, it must leap over the *else* path, because otherwise execution would fall straight into code meant only for the false case. That **unconditional jump at the tail of the first branch, landing on the same block the second branch falls into**, is the distinguishing mark of an `if/else` as opposed to a bare `if`. Both branches reunite at `loc_merge`, where the shared tail (`C = 23`, i.e. `17h`) runs.



4.3 The `while` loop — `fun_while`

Loops reveal a second compiler habit: the **condition test is placed at the bottom** of the loop, and the function jumps straight to that test on entry.

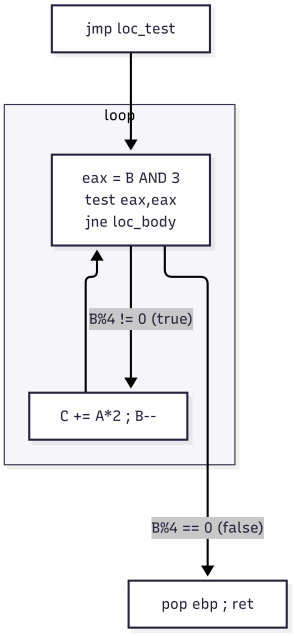
```
void fun_while() {
    while (B % 4 != 0) {
        C += A * 2;
        B--;
    }
}
```

```
push    ebp
mov     ebp, esp
jmp     short loc_test      ; jump straight to the condition
loc_body:
mov     eax, ds:A
lea     edx, [eax+eax]      ; edx = A * 2
mov     eax, ds:C
add     eax, edx
mov     ds:C, eax           ; C = C + A*2
mov     eax, ds:B
sub     eax, 1
mov     ds:B, eax           ; B = B - 1
loc_test:
mov     eax, ds:B
and     eax, 3              ; eax = B % 4   (keep the low two
bits)
test    eax, eax            ; is the remainder zero?
jne     short loc_body      ; if non-zero, loop again
nop
pop     ebp
ret
```

Three idioms appear together here. First, the structure is **test-at-the-bottom with an initial jump to the test**: the very first thing the function does is `jmp loc_test`, so the condition is evaluated before the first iteration, exactly matching `while` semantics (the body may run zero times). The back-edge is the conditional jump `jne loc_body` that climbs back up to the body when the loop should continue. A backward-pointing conditional jump is the universal signature of a loop.

Second, `B % 4` is computed not with a division instruction but with `and eax, 3`. Because four is a power of two, the remainder modulo four is simply the value of the **low two bits**, and `and` with the mask `3` (binary `11`) isolates exactly those bits. The subsequent `test eax, eax` checks whether that result is zero. This "mask with (2^n − 1) to take a modulo by 2^n" trick is extremely common and should be read directly as a modulo operation.

Third, `A * 2` is computed with `lea edx, [eax+eax]` rather than a multiply — the subject of §5.



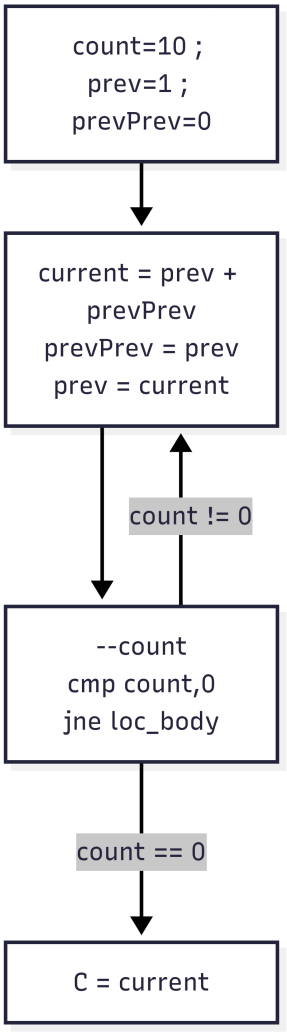
4.4 The `do / while` loop — `fun_do_while`

A `do/while` differs from a `while` in exactly one structural way: there is **no initial jump to the test**, because the body must always run at least once.

```
void fun_do_while() {
    int count = 10;
    int current, prev = 1, prevPrev = 0;
    do {
        current = prev + prevPrev;
        prevPrev = prev;
        prev = current;
    } while (--count);
    C = current;
}
```

```
sub     esp, 10h
mov     [ebp-4], 0Ah        ; count = 10
mov     [ebp-8], 1          ; prev = 1
mov     [ebp-0Ch], 0        ; prevPrev = 0
loc_body:
mov     edx, [ebp-8]        ; prev
mov     eax, [ebp-0Ch]      ; prevPrev
add     eax, edx
mov     [ebp-10h], eax      ; current = prev + prevPrev
mov     eax, [ebp-8]
mov     [ebp-0Ch], eax      ; prevPrev = prev
mov     eax, [ebp-10h]
mov     [ebp-8], eax        ; prev = current
sub     [ebp-4], 1          ; --count
cmp     [ebp-4], 0
jne     short loc_body      ; while (count != 0)
mov     eax, [ebp-10h]
mov     ds:C, eax           ; C = current
```

Execution enters the body directly; only after the body does it decrement `count` and test it with a backward `jne`. The absence of a leading `jmp` to the condition is precisely what separates `do/while` from `while` at the assembly level. This routine also illustrates how multiple locals are packed into the frame — `count`, `prev`, `prevPrev`, and `current` occupy `[ebp-4]`, `[ebp-8]`, `[ebp-0Ch]`, and `[ebp-10h]` — and how a classic Fibonacci-style sliding-window computation looks once reduced to register shuffling.



4.5 The `for` loop — `fun_for`

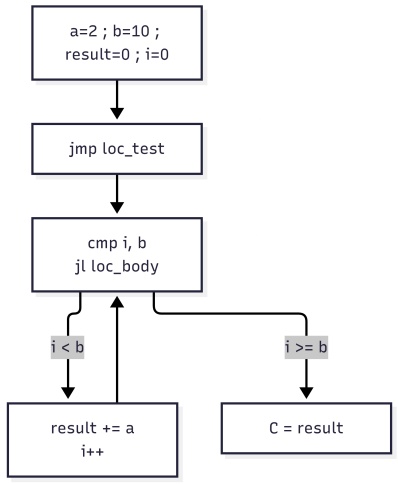
A `for` loop is, at the machine level, a `while` loop with its four parts (initialization, condition, body, increment) laid out in a fixed arrangement:

```
void fun_for() {
    int a = 2, b = 10;
    int i, result = 0;
    for (i = 0; i < b; i++) {
        result += a;
    }
    C = result;
}
```

```
sub     esp, 10h
mov     [ebp-0Ch], 2        ; a = 2
mov     [ebp-10h], 0Ah      ; b = 10
mov     [ebp-8], 0          ; result = 0
```

```
mov     [ebp-4], 0          ; i = 0           (initialization)
jmp     short loc_test      ; jump to the condition
loc_body:
mov     eax, [ebp-0Ch]
add     [ebp-8], eax        ; result += a     (body)
add     [ebp-4], 1          ; i++             (increment)
loc_test:
mov     eax, [ebp-4]
cmp     eax, [ebp-10h]
jl      short loc_body      ; i < b ?         (condition)
mov     eax, [ebp-8]
mov     ds:C, eax           ; C = result
```

The layout matches the `while` template exactly — an initial `jmp` to a bottom-placed test, a backward conditional jump as the back-edge — with one addition: the **increment is emitted at the end of the body**, right before control reaches the test. Recognizing that the last statement of the loop body is a counter update (`add [ebp-4], 1`) separated from the user's actual work (`result += a`) is what lets an analyst reconstruct a `for` rather than an indistinct `while` . Note also that the condition here is `i < b` , and the emitted jump is `jl` (jump if less) — a case where the compiler kept the condition *un-inverted* because the jump target is the body rather than the exit. Whether the condition is inverted depends on whether the jump guards the body or skips it, and the analyst determines that from the jump's target, not from a fixed rule.



5. The `LEA` Instruction

Several of the routines above compute values with `lea` , an instruction whose behavior is a frequent source of confusion and therefore a favorite of both

compilers and obfuscators.

`LEA` **(Load Effective Address)** evaluates a memory-address expression and writes the resulting *address* into a destination register. The essential distinction is this: ordinary data-movement and arithmetic instructions that use a memory operand **dereference** it — they read or write the contents of the pointed-to location. `LEA` does **not** touch memory at all. It performs only the address *calculation* implied by the brackets and stores that number. In other words, where `mov eax, [ebx+ecx*4]` fetches the value *at* `ebx + ecx*4`, `lea eax, [ebx+ecx*4]` merely computes `ebx + ecx*4` and keeps it.

This gives `LEA` two distinct uses:

The first is its literal purpose, **address calculation for pointer arithmetic** — computing the address of an array element or structure field without reading it, for example to pass a pointer to another function.

The second, and the reason it appears in arithmetic-heavy code, is as a **fast general-purpose arithmetic instruction**. Because the x86 addressing mode can encode `base + index*scale + displacement` in a single operation — with scale factors of 1, 2, 4, or 8 — `LEA` can evaluate small linear expressions in one instruction, without disturbing the flags register. Thus `lea edx, [eax+eax]` computes `eax * 2` (seen in `fun_while` for `A * 2`), `lea eax, [ecx+ebx]` computes the sum `ecx + ebx`, and `lea eax, [eax+eax*4]` computes `eax * 5`. When an analyst sees `LEA` with an arithmetic-looking bracket expression and the result is subsequently used as a *number* rather than as a pointer, it should be read as plain arithmetic. Confusing the two readings — treating a computed constant as a pointer, or vice versa — is one of the most common early mistakes in reading disassembly.

6. Analyst's Reference: Pattern-to-Construct Cues

The following cues summarize the fingerprints established in this chapter. They are stated in terms that survive stripping, since in real analysis the names are gone.

A **prologue** `push ebp` / `mov ebp, esp` (optionally followed by `sub esp, N`) marks a function entry; a matching `leave` / `ret` or `pop ebp` / `ret` marks its exit. Operands of the form `[ebp - k]` are **local variables**; operands `[ebp + 8]`, `[ebp + 0Ch]`, `[ebp + 10h]` are **incoming arguments**; operands that are **absolute addresses** are **globals**.

A `cmp` or `test` followed by a **conditional jump** is a decision point. If the jump **skips a block** and both paths reconverge, it is an **`if`**; the tested condition is the *inverse* of the source condition. If the first block ends in an **unconditional `jmp` over a second block** and both reconverge, it is an **`if/else`**. A **backward** conditional jump is a **loop back-edge**: if the function **jumps to the condition first** and the body is below, it is a **`while`** (or a `for`, if the body's last action is a counter increment); if the body runs **before** any test, it is a **`do/while`**.

Arithmetic shortcuts recur constantly: `and reg, (2^n - 1)` is a **modulo by 2^n**; a `shl` / `shr` is a multiply/divide by a power of two; and `lea` with a bracketed expression used as a number is **linear arithmetic**, not a memory access. Hexadecimal constants should be converted on sight (`0Ah` = 10, `14h` = 20, `1Eh` = 30).

Finally, and most importantly: the symbol names, inferred prototypes, and structural comments supplied by the disassembler are **hypotheses to verify, not facts**. In a genuine investigation they will be absent or deliberately misleading, and the only reliable ground truth is the instruction stream itself.